

Modulo Cuestionarios

Documentacion tecnica — ISMP 3D Anatomy Backend (NestJS)
Fecha: 16 de junio de 2026

1. Descripcion general

El modulo **CuestionariosModule** es el nucleo academico del backend. Gestiona la creacion, consulta, actualizacion y eliminacion de cuestionarios de anatomia, junto con sus preguntas anidadas. Esta construido sobre NestJS con Prisma ORM y expone una API REST bajo el prefijo `/cuestionarios`.

Archivos del modulo

Archivo	Responsabilidad
cuestionarios.module.ts	Declara e importa el modulo en NestJS
cuestionarios.controller.ts	Define los endpoints HTTP y aplica guards
cuestionarios.service.ts	Logica de negocio y acceso a la base de datos via Prisma
dto/create-cuestionario.dto.ts	Forma esperada del body para crear un cuestionario
dto/update-cuestionario.dto.ts	Forma esperada del body para actualizar (campos opcionales)

2. Cambios introducidos en esta sesion

2.1 Guards movidos al nivel de clase

Originalmente, los endpoints **GET /cuestionarios** y **GET /cuestionarios/:id** eran publicos (sin guards), permitiendo el acceso sin autenticacion. Los endpoints de escritura (POST, PATCH, DELETE) si tenian los guards declarados individualmente en cada metodo.

Cambio introducido: Se movio `@UseGuards(JwtGuard, RolesGuard)` al nivel del decorador de clase `@Controller`, haciendo que todos los endpoints requieran token JWT valido. Los `@Roles` se mantienen en los metodos que los necesitan.

Antes del cambio (fragmento):

```
// GET sin guard – cualquiera podia acceder @Get() findAll(@Query('materiaId')
materiaId?: string) { ... } // POST con guard individual @UseGuards(JwtGuard,
RolesGuard) @Roles(Role.DOCENTE, Role.ADMIN) @Post() create(...) { ... }
```

Despues del cambio:

```
@UseGuards(JwtGuard, RolesGuard) // aplica a TODOS los endpoints
@Controller('cuestionarios') export class CuestionariosController { @Get() //
solo JWT, sin restriccion de rol findAll(...) { ... } @Roles(Role.DOCENTE,
Role.ADMIN) // JWT + rol (del guard de clase) @Post() create(...) { ... } }
```

2.2 Registro del modulo en AppModule

El modulo existia en el sistema de archivos pero no estaba importado en AppModule, por lo que NestJS no lo cargaba y los endpoints no estaban disponibles en la API.

Cambio introducido: Se agrego CuestionariosModule al array imports de AppModule. A partir de este cambio los endpoints /cuestionarios estan activos en el servidor.

```
// backend/src/app.module.ts import { CuestionariosModule } from
'./cuestionarios/cuestionarios.module'; @Module({ imports: [
ConfigModule.forRoot({ isGlobal: true }), PrismaModule, AuthModule,
UsersModule, CuestionariosModule, // <-- agregado ], }) export class AppModule
{}
```

3. Endpoints disponibles

Todos los endpoints requieren token JWT valido (Bearer token en el header Authorization). Los endpoints de escritura ademas exigen rol **DOCENTE** o **ADMIN**.

Metodo	Ruta	Guard JWT	Roles requeridos	Descripcion
GET	/cuestionarios	Si	Cualquier rol	Lista de cuestionarios (filtro opcional por materialid)
GET	/cuestionarios/:id	Si	Cualquier rol	Detalle del cuestionario con preguntas ordenadas
POST	/cuestionarios	Si	DOCENTE, ADMIN	Crea cuestionario con preguntas anidadas
PATCH	/cuestionarios/:id	Si	DOCENTE, ADMIN	Actualiza. Reemplaza todas las preguntas si se envian
DELETE	/cuestionarios/:id	Si	DOCENTE, ADMIN	Elimina. Solo el autor o ADMIN pueden borrar

4. Modelo de autorizacion en dos capas

El modulo implementa autorizacion en dos niveles independientes para separar la responsabilidad de validacion HTTP de la logica de negocio:

Capa	Responsable	Que verifica	Si falla
1 - HTTP	JwtGuard + RolesGuard	Token valido y rol suficiente	401 / 403 inmediato
2 - Negocio	CuestionariosService	Es el autor del cuestionario o es ADMIN	ForbiddenException (403)

Nota: El guard de roles solo verifica si el usuario tiene un rol suficiente para intentar la operacion. La verificacion de ownership (si el docente es el autor del cuestionario que quiere modificar) la hace el service, no el guard.

5. Logica clave del CuestionariosService

5.1 Creacion con preguntas anidadas

Al crear un cuestionario, las preguntas se insertan en la misma operacion Prisma usando la relacion anidada `preguntas.create`. Esto garantiza atomicidad: si alguna pregunta falla, el cuestionario tampoco se crea.

```
prisma.cuestionario.create({ data: { titulo, descripcion, materiaId, formato,
autorId, preguntas: { create: dto.preguntas.map((p, i) => ({ orden: i, texto:
p.texto, opciones: p.opciones, correct: p.correcta, explicacion: p.explicacion
?? '', })), }, }, })
```

5.2 Actualizacion con replace-all de preguntas

Si el DTO de actualizacion incluye el campo `preguntas`, la estrategia es **reemplazar todas**: primero se borran las preguntas actuales con `deleteMany: {}` y luego se crean las nuevas. No existe edicion granular por pregunta individual.

```
preguntas: { deleteMany: {}, // borra todas las preguntas actuales create:
preguntas.map((p, i) => ({ orden: i, ...p })), }
```

Nota: Esta estrategia simplifica el codigo pero invalida cualquier `attemptId` o log que referenciara preguntas anteriores por su ID. A considerar cuando se implemente el modulo de intentos (Attempt).

5.3 Verificacion de ownership

Tanto en **update** como en **remove**, el service verifica existencia primero y luego ownership antes de ejecutar la operacion:

```
if (existing.autorId !== requesterId && requesterRole !== Role.ADMIN) { throw
new ForbiddenException( 'Solo el autor o un administrador puede modificar este
cuestionario', ); }
```

6. Estado actual del modulo

Aspecto	Estado	Notas
Endpoints CRUD	Completo	GET, POST, PATCH, DELETE implementados

Autenticacion	Completo	JwtGuard en todos los endpoints
Autorizacion por rol	Completo	RolesGuard + @Roles en escritura
Ownership check	Completo	Validado en service para PATCH y DELETE
Registro en AppModule	Completo	Modulo activo en el servidor
Modulo Attempts	Pendiente	Sin controller ni service aun
Tests	Pendiente	No hay test runner configurado en el proyecto